Open PDF RAG — Project Guide

A local, source-cited question-answering system for an astronomy PDF library

This guide documents the implementation, the supplied corpus, how to run it, what has and has not been measured, and the project's limitations. It describes the current code and files; it does not claim that a vector index has already been built.

1. Project at a glance

The application is a Retrieval-Augmented Generation (RAG) system. It takes PDFs from a local folder, extracts page text (using OCR when needed), splits the text into page-attributed passages, embeds those passages, and saves them in a persistent vector index. At question time, it embeds the question, retrieves similar passages, and asks a local language model to answer from those passages. The interface shows the source filename, PDF page, similarity score, and excerpt alongside each answer.

Requirement	Implementation
PDF text	PyMuPDF native text extraction
Scanned pages / embedded raster images	Tesseract OCR through `pytesseract`
Chunking	Deterministic 650-word chunks, 130-word overlap, page-scoped
Embeddings	`sentence-transformers/all-MiniLM-L6-v2`
Vector index	Persistent Chroma collection using cosine HNSW
Answer generation	Local Ollama server; default model `qwen2.5:3b`
User interface	FastAPI JSON API with a small browser chat UI
Sources	Filename, 1-based PDF page number, score and retrieved excerpt
Evaluation	Query mean/p95 latency, filename Recall@k and MRR script

The embedding model and vector database are open-source software. Ollama serves a local model; no remote generation API is called by the application. The model files are downloaded on first use and require local disk and memory.

2. Supplied corpus

The provided directory contains 10 astronomy PDFs with **3,900 pages total**. Every file has at least 200 pages, satisfying the dataset-size requirement. The PDFs have been copied to `data/` for the application. The original source directory, `Astronomy_RAG_PDFs/`, is preserved. Neither source copy is committed to Git by default.

File	Pages

`01_Introduction_to_Astronomy.pdf`	601
`02_Romance_of_Modern_Astronomy.pdf`	424
`03_Treatise_for_Colleges.pdf`	339
`04_Theoretical_and_Descriptive_Astronomy.pdf`	284
`05_Elements_of_Astronomy_Rolfe.pdf`	380
`06_Story_of_the_Stars.pdf`	356
`07_Compendium_of_Astronomy.pdf`	317
`08_Navigation_and_Nautical_Astronomy.pdf`	240
`09_Astronomy_of_the_Bible.pdf`	442
`10_Elements_of_Astronomy_Young.pdf`	517
Total	**3,900**

A native-text scan found fewer than 40 non-whitespace characters on 2,270 pages. Those pages take the full-page OCR path. Other pages can still contain raster images, which the implementation OCRs separately when they are large enough. This figure estimates pages that need OCR based on extracted text; it is not an OCR accuracy measurement.

3. Processing pipeline

Ingestion

1. The `/api/ingest` endpoint scans `RAG_DATA_DIR` recursively for PDFs. 2. PyMuPDF extracts text blocks and page counts. Short-text pages are rendered and sent to Tesseract OCR. Large embedded images on text-bearing pages are also OCRed. 3. Repeated lines found in page-edge blocks are removed from native text. Language is detected per page with a deterministic language detector. 4. Text is split independently per page into chunks of 650 whitespace-delimited words with a 130-word overlap. Very short passages (under 20 words) are skipped. Each chunk retains its filename, PDF page, language, file hash, source ID, and chunk number. 5. The local embedding model converts passage text to normalized vectors. Chroma stores the vectors, text, and metadata persistently using cosine distance and its HNSW approximate-nearest-neighbor index. 6. A JSON manifest records the processed source files and their page/chunk counts. Re-ingesting a PDF replaces its previous chunks.

Ingestion runs as a FastAPI background task. The UI polls status every three seconds. OCR and vector generation across 3,900 pages can take substantial time; it is an offline preparation stage, not part of typical query latency.

Question answering

1. The user submits a question to `POST /api/chat`. 2. The same embedding model embeds the question. 3. Chroma retrieves the top passages by approximate cosine similarity. 4. The service sends the question and labeled passages to local Ollama. The prompt instructs the model to answer from the excerpts and cite labels. 5. The API returns the answer, source passages, similarity values, response mode, and observed duration. The UI shows the citations and excerpts.

If Ollama is unavailable or returns an invalid response, the service returns the most relevant passage as an explicitly labeled **extractive fallback**. That fallback is not presented as an LLM-generated answer.

4. Running the application

Recommended: Docker Compose

Requirements: Docker Desktop / Docker Engine running, internet for initial model downloads, and enough local memory, storage, and compute.

```
docker compose up --build -d
docker compose exec ollama ollama pull qwen2.5:3b
```

Open http://localhost:8000, select **Index PDFs in ./data**, and wait until the progress indicator shows completion. Then enter a question. The first container/model start downloads the Python packages, embedding model, and Ollama generation model. `data/` and `vector_db/` are bind-mounted so inputs and generated index persist locally. Ollama models persist in a Docker volume.

The Docker image installs English Tesseract language data. For other languages, install the required Tesseract language packs and configure `OCR_LANG`.

Local Python setup

Use Python 3.11 or later, Tesseract OCR and the language data, plus a local Ollama installation. Install the Python requirements and start the model and web service:

```
python -m pip install -r requirements.txt
ollama pull qwen2.5:3b
uvicorn app:app --reload
```

Put PDFs in `data/`. The defaults can be changed with `RAG_DATA_DIR`, `RAG_DB_DIR`, `EMBEDDING_MODEL`, `OLLAMA_URL`, `OLLAMA_MODEL`, and `OCR_LANG`. See `.env.example` for the supported environment variable names; the app process needs those variables set in its environment.

5. API and observability

Endpoint	Purpose
`GET /api/health`	Liveness check
`GET /api/status`	Ingested PDFs, pages, chunks, target coverage, progress
`POST /api/ingest`	Start ingestion of PDFs already in the configured folder
`POST /api/chat`	Retrieve passages and answer a question
`GET /api/metrics`	In-process query count, mean/p95 latency and fallback count

Example request:

```
{"question": "How does the text describe the phases of the Moon?", "top_k": 5}
```

The chat response includes the answer, response mode, observed end-to-end milliseconds, and sources with `filename`, 1-based `page`, `score`, and excerpt. The status endpoint reports whether at least 10 indexed PDFs meet the 200-page minimum. Metrics are in-process and reset when the service restarts.

`evaluate.py` accepts a JSON benchmark of questions and relevant filenames. It reports filename-

based Recall@k, MRR, and mean/p95 request latency. The metrics are useful for comparisons but do not by themselves validate factual correctness, citation entailment, or OCR accuracy.

6. Tests and measured results

Two unit tests cover deterministic chunk size/overlap and empty-input behavior. Python compilation and both tests passed in the development environment. Editor diagnostics reported no errors.

The **input corpus size is verified** (10 PDFs; 3,900 pages), but at the time this guide was written:

- The vector index had **not** been built; no indexed passage count is claimed.
- OCR quality and extraction correctness had **not** been manually evaluated.
- The RAG API had **not** been exercised end to end.
- No question latency, p95 latency, relevance, hallucination, or citation

accuracy score had been measured.

The 2–5 second objective is hardware- and model-dependent. Populate a labeled benchmark, ingest the PDFs, warm up both models, and measure on the intended machine before claiming that target.

7. Repository and data safety

The `.gitignore` excludes `data/`, `Astronomy_RAG_PDFs/`, and generated `vector_db/` contents. Keep the source corpus and derived vectors out of a public repository unless you have checked their license and explicitly intend to publish them. The project guide and source code can be versioned separately from the corpus.

The demo has no user authentication or authorization. Keep it on a trusted local machine; do not expose the API to untrusted networks without adding access control, rate limits, and deployment hardening. The ingestion API only reads the configured local directory and does not accept paths from callers.

8. Project files

- `app.py` — FastAPI service, extraction, OCR, ingestion, retrieval, generation,

status and metrics.

- `chunking.py` — deterministic chunking rules.
- `web/index.html` — browser chat UI and status/metrics display.
- `evaluate.py` — labeled-retrieval benchmark runner.
- `tests/test_chunking.py` — chunking unit tests.
- `requirements.txt` — Python dependencies.
- `Dockerfile` / `compose.yaml` — containerized app and local Ollama service.
- `README.md` — quick start and operational reference.
- `PROJECT_GUIDE.pdf` — this guide as a portable document.